

Báo cáo

Bài 5. Tìm hiểu về Windows API

MỤC LỤC

1. Lý thuyết.....	3
1.1 Định nghĩa và chức năng.....	3
1.2 Các WinAPI mã độc thường sử dụng.....	3
2. Thực hành.....	5
2.1 Thực hiện.....	5
2.2. Kết quả.....	7

1. Lý thuyết

1.1 Định nghĩa và chức năng

Định nghĩa: Windows API (hay Win32 API) là nền tảng thuần (native platform) của các ứng dụng Windows.

Chức năng:

- WinAPI là sự lựa chọn tốt nhất đối với các ứng dụng cần quyền truy cập trực tiếp vào phần cứng và các tính năng của hệ thống, ứng dụng yêu cầu hiệu năng cao và tối ưu ở mức phần cứng.
- WinAPI có thể được dùng bởi mọi ứng dụng desktop, hỗ trợ cả phiên bản 32-bit và 64-bit.

Nguồn tham khảo: [Microsoft](#)

1.2 Các WinAPI mã độc thường sử dụng

Nguồn: [MalAPL.io](#),

VirtualAlloc

- Thuộc thư viện memoryapi.h và kernel32.dll.
- Dùng để bảo lưu, cấp phát, hoặc thay đổi trạng thái của một vùng các trang trong không gian địa chỉ ảo của tiến trình gọi. Vùng nhớ được cấp phát bởi hàm này sẽ tự động được khởi tạo về giá trị 0.
- Là 1 đặc trưng của kỹ thuật [Process Injection](#).

CreateRemoteThread

- Thuộc thư viện processthreadsapi.h và kernel32.dll.
- Dùng để tạo 1 luồng thực thi trong không gian địa chỉ ảo của 1 tiến trình khác.
- Là 1 đặc trưng của kỹ thuật Process Injection.

CreateFileMapping

- Thuộc thư viện winbase.h và kernel32.dll.
- Dùng để tạo một định danh/handle cho một ánh xạ tệp (file mapping) nhằm tải một tệp vào bộ nhớ và giúp người dùng có thể truy cập tệp đó thông qua các địa chỉ bộ nhớ. Các chương trình khởi chạy (launcher), trình nạp (loader) và trình tiêm mã (injector) sử dụng hàm này để đọc và chỉnh sửa các tệp PE.
- Được dùng trong Process Injection.

ShellExecute

- Thuộc thư viện shellapi.h và shell32.dll.
- Dùng để thực hiện 1 hành động lên 1 file cụ thể.

RegSetValue

- Thuộc thư viện winreg.h và advapi32.dll.

- Dùng để thiết lập dữ liệu cho 1 giá trị mặc định/chưa được đặt của một registry key cụ thể. Kiểu dữ liệu là chuỗi ký tự đọc được.
- Có thể là dấu hiệu của persistence.

2. Thực hành

Đề bài: Viết chương trình C/C++ thực hiện đọc 1 file path từ console -> mở file từ file path đó -> đọc file đó -> in ra console nội dung của file (tối đa 100 ký tự đầu). Không sử dụng std::out hay printf để in ra console.

2.1 Thực hiện

Sử dụng C++.

Sử dụng hàm main thay vì WinMain để có hiển thị cửa sổ command line console thay vì GUI.

Sử dụng các biến: chBuffer để làm buffer lưu trữ dữ liệu trung gian, cRead để lưu số byte đã đọc.

Tạo handle hStdin, hStdout để thao tác với input, output.

Phương pháp debug: ([tham khảo](#)) hiển thị messagebox báo lỗi. Đặt messagebox báo lỗi sau mỗi thao tác.

Compile trên Linux: x86_64-w64-mingw32-g++ test.cpp -o test.exe -mconsole.

a. Đọc file path từ console

Sử dụng API [ReadFile](#), tham khảo ví dụ [tại đây](#).

Vì ReadFile là 1 hàm BOOL, ta có thể thực hiện trực tiếp ReadFile với các tham số theo template của Microsoft:

- hStdin: handle đến input.
- chBuffer: buffer lưu dữ liệu đọc được từ input.
- Đọc tối đa 255 bytes (tùy chỉnh theo khai báo chBuffer).
- &cRead: con trỏ đến địa chỉ lưu số byte đã đọc.
- Chương trình đơn luồng nên giá trị cho lpOverlapped là NULL.

Trim đi các ký tự xuống dòng (\n), (\r\n).

```
int main(){
    CHAR chBuffer[256];
    DWORD cRead;

    hStdin = GetStdHandle(STD_INPUT_HANDLE);
    hStdout = GetStdHandle(STD_OUTPUT_HANDLE);

    if(!ReadFile(
        hStdin,
        chBuffer,
        255,
        &cRead,
        NULL))
    {
        MessageBox(NULL, TEXT("Error reading file path"), TEXT("Console Error"), MB_OK);
        return 1;
    };

    chBuffer[cRead]='\0';
    chBuffer[strcspn(chBuffer, "\r\n")]='\0';
}
```

b. Mở file từ file path đã đọc

Sử dụng API [CreateFile](#).

(CreateFile thực chất là 1 macro trỏ đến CreateFileA/CreateFileW/CreateFileEx. Chương trình này thao tác với kiểu dữ liệu char tiêu chuẩn nên tự động trỏ đến CreateFileA.)

Vì CreateFile trả về 1 handle, nên cần tạo 1 handle hFile làm biến trả về.

Thực hiện mở file bằng CreateFile với các tham số:

- Dữ liệu đọc vào là chBuffer đang lưu file path từ input ở mục a.
- Cấp quyền chỉ đọc (tùy chọn).
- lpSecurityAttributes để là NULL vì hiện không cần chia sẻ với tiến trình con.
- Disposition: OPEN_EXISTING
- Sử dụng các thuộc tính mặc định.
- Không sử dụng template file nên tham số cuối đặt là NULL.

```
hFile = CreateFile(  
    chBuffer,  
    GENERIC_READ,  
    FILE_SHARE_READ,  
    NULL,  
    OPEN_EXISTING,  
    FILE_ATTRIBUTE_NORMAL,  
    NULL  
);  
if (hFile == INVALID_HANDLE_VALUE)  
{  
    MessageBox(NULL, TEXT("Error opening file"), TEXT("Console Error"), MB_OK);  
    return 1;  
}
```

c. Đọc nội dung file

Sử dụng API ReadFile với các tham số:

- Đọc từ hFile - handle của file được mở ở mục b.
- Lưu dữ liệu đọc được, ghi đè vào chBuffer.
- Đọc tối đa 100 bytes.
- Dùng &cRead để lưu lượng bytes đã đọc.
- Không overlap (NULL).

```
if(!ReadFile(  
    hFile,  
    chBuffer,  
    100,  
    &cRead, //reuse ok bc the old version will be overwritten  
    NULL))  
{  
    MessageBox(NULL, TEXT("Error reading file content"), TEXT("Console Error"), MB_OK);  
    return 1;  
};
```

d. In ra console nội dung file đã đọc

Sử dụng API [WriteFile](#), tham khảo [ví dụ](#).

Vì WriteFile là 1 hàm BOOL, nên ta có thể thực hiện trực tiếp tương tự ReadFile, với các tham số:

- In ra output: sử dụng handle hStdout.
- Dùng dữ liệu đang lưu trong chBuffer để in.
- Viết ra số lượng byte là: cRead - đang lưu số lượng byte đã đọc từ thao tác ở mục c.
- Lưu lượng byte đã in ra, ghi đè vào cRead.
- Không overlap (NULL).

```

if(!WriteFile(
    hStdout,
    chBuffer,
    cRead,
    &cRead,
    NULL))
{
    MessageBox(NULL, TEXT("Error printing out file content"), TEXT("Console Error"), MB_OK);
    return 1;
};

```

Cuối cùng, giải phóng handle hFile đã tạo (hStdin/out là các handle của hệ thống nên không cần giải phóng thủ công). Đồng thời, vì kết quả của mục d. sau khi được in ra sẽ đi tiếp return 0; và quit luôn console, có thể dùng ReadFile thêm lần nữa (sử dụng đọc từ hStdin - chờ input từ người dùng - giúp tạm dừng chương trình) để dễ quan sát kết quả in ra hơn.

```

CloseHandle(hFile);
DWORD last;
ReadFile(
    hStdin,
    chBuffer,
    1,
    &last,
    NULL
);
return 0;

```

2.2. Kết quả

Sau khi biên dịch, click đúp chuột để chạy file test.exe, 1 cửa sổ command line console sẽ xuất hiện. Nếu nhập vào 1 file path tồn tại, chương trình sẽ trả về 100 ký tự đầu tiên trong nội dung của file.

